

STUDENT LEARNING MODULE

IoT Prototyping

Using Raspberry Pi Zero

MODULE THREE

PRACTICAL SESSION

From a working prototype to a browser-based IoT dashboard

MODULE OVERVIEW

GitHub, FastAPI dashboard and remote access

Clone the training repository, prepare the application environment, run FastAPI and access the dashboard through Tailscale.

Learning outcomes

- Explain the purpose of Git, GitHub, FastAPI routes and API methods.
- Clone and inspect the approved training repository.
- Create and activate the Python virtual environment.
- Verify GPIO Zero and smbus2 compatibility with the Module Two hardware.
- Run FastAPI and test health and sensor API routes.
- Display live sensor readings and control the pump through the dashboard.
- Access and troubleshoot the complete prototype through Tailscale.

Learning sequence

Stage	Activity
Review	Confirm the completed Module Two prototype and recorded values
Retrieve	Clone and inspect the approved GitHub repository
Prepare	Create the virtual environment and install requirements
Configure	Enter the group hardware and calibration values
Run	Start FastAPI and verify local API routes
Access	Open the dashboard through the authorised Tailscale connection
Validate	Test sensors, bounded pump control and safe shutdown

Equipment and Software

Requirement	Details
Raspberry Pi	Completed Module Two prototype
Sensors	SHT31, ADS1115 and capacitive soil-moisture sensor
Actuator	3.3 V-compatible relay, LED and externally powered DC pump
Student device	Laptop with Tailscale installed and signed in
Software	Python 3, Git, FastAPI, Uvicorn, GPIO Zero and smbus2
Repository	Instructor-approved training GitHub repository

Safety reminder

Disconnect external pump power before changing relay wiring. Keep water away from the Raspberry Pi and breadboard.
Confirm the pump is OFF before starting the web application. Never use mains voltage in this practical.

Required starting condition

Use the same assigned Raspberry Pi, hostname and personal Tailscale account from Modules One and Two.
The SHT31 and ADS1115 must appear on I2C, calibration values must be available, and the relay must remain OFF.

Understanding Git and GitHub

Git is a version-control system used to track and manage source-code changes. GitHub is an online platform that stores Git repositories.

Term	Meaning
Repository	A project folder managed by Git
Clone	Create a local copy of a repository
Commit	Record a set of changes
Pull	Download newer changes
Push	Upload local changes
Branch	A separate line of development
README	Project description and instructions
requirements.txt	List of Python packages

Install Git

```
sudo apt update
sudo apt install -y git
git --version
```

Clone the Project

```
cd ~
git clone https://github.com/INSTRUCTOR/iot-pi-training.git
cd iot-pi-training
ls -la
```

Repository address

Replace the example URL with the repository address provided for the training.

Expected Repository Structure

```

iot-pi-training/
|
|-- README.md
|-- requirements.txt
|-- .gitignore
|
|-- session1/
|   |-- led_control.py
|
|-- session2/
|   |-- read_sht31.py
|   |-- read_soil.py
|   |-- pump_control.py
|   |-- automatic_irrigation.py
|
'-- dashboard/
    |-- main.py
    |-- hardware.py
    |-- config.py
    |-- templates/
    |   |-- index.html
    '-- static/
        |-- app.js
        '-- style.css

```

Inspect the Repository

```

cat README.md
ls -la dashboard
cat requirements.txt
git status

```

Update an Existing Repository

```
git pull
```

Repository Check

Task	Completed
Git is installed.	
Repository is cloned.	
README is opened.	
Dashboard folder is located.	
requirements.txt is located.	
Project structure is understood.	

Preparing the Python Environment

The dashboard virtual environment isolates its web dependencies while reusing the verified Module Two hardware code.

```
cd ~/iot-pi-training/dashboard
python3 -m venv --system-site-packages .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install -r requirements.txt
```

Check installed packages

```
python -c "import fastapi, uvicorn; print('Web libraries available')"
```

```
python -c "import gpiozero, smbus2; print('Hardware libraries available')"
```

```
python -c "from read_soil import read_adc; print('Module Two sensor code available')"
```

Environment check

Task	
Dashboard folder is open.	
Virtual environment uses system site packages.	
Requirements install successfully.	
FastAPI and Uvicorn import successfully.	
GPIO Zero and smbus2 import successfully.	
Module Two sensor helper imports successfully.	

Understanding the FastAPI Application

FastAPI connects the physical prototype to a browser. It reads sensors, provides API responses, displays the dashboard and receives pump-control commands.

```

Browser
  |
  | HTTP request
  v
FastAPI application
  |-- Read SHT31
  |-- Read ADS1115
  |-- Control LED
  '-- Control relay and pump
  
```

File	Purpose
main.py	FastAPI routes and application startup
hardware.py	Sensor and actuator functions
config.py	Calibration and GPIO settings
index.html	Dashboard page structure
app.js	Live updates and button actions
style.css	Dashboard appearance

API Routes

Route	Method	Purpose
/	GET	Display dashboard
/health	GET	Check application status
/api/readings	GET	Return sensor readings
/api/pump/on	POST	Turn pump on
/api/pump/off	POST	Turn pump off
/api/pump/timed	POST	Run pump for a limited duration

Configure the Hardware Values

Open `config.py` and enter the calibration and GPIO values recorded in Module Two.

```
nano config.py

DRY_VALUE = 22000    # replace with measured value
WET_VALUE = 9000     # replace with measured value
LED_PIN = 17    RELAY_PIN = 27    SAFE_PUMP_DURATION = 3
```

Record the active configuration

Setting	Your value
Dry calibration value	
Wet calibration value	
LED GPIO	17
Relay GPIO	27
Relay active state	
Safe pump duration	3 seconds

Run the FastAPI application

```
source ~/iot-pi-training/dashboard/.venv/bin/activate
cd ~/iot-pi-training/dashboard
uvicorn main:app --host 0.0.0.0 --port 8000
```

Network safety

Binding to 0.0.0.0 listens on every Raspberry Pi interface. Use only the approved private training network, with the instructor's Tailscale access policy. Never expose this dashboard directly to the public internet.

Test the Application Locally

```
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/api/readings
```

Expected health response:

```
{"status": "running", "application": "IoT Dashboard"}
```

Local Test Check

Task	Completed
FastAPI starts without an error.	
Health route returns a response.	
Sensor route returns data.	
Temperature is included.	
Humidity is included.	
Soil moisture is included.	
Pump status is included.	

Access Through Tailscale

```
tailscale ip -4
```

Open the dashboard on an authorised laptop or mobile device:

```
http://TAILSCALE-IP:8000
```

Open the interactive API documentation:

```
http://TAILSCALE-IP:8000/docs
```

Where MagicDNS is enabled, you may also use:

```
http://iot-set03:8000
```

Dashboard Access Record

Access method	Address
Tailscale IP	
MagicDNS hostname	
Dashboard	
API documentation	
Health route	

Understanding the Data Flow

```

SHT31 and soil sensor
  |
  v
hardware.py
  |
  v
/api/readings
  |
  v
app.js
  |
  v
Browser dashboard

```

Dashboard Observation Record

Test	Before	After
Temperature		
Humidity		
Dry-soil reading		
Wet-soil reading		
Pump status		

Practical Activity: Pump Control

Manual Control

Use the dashboard Pump ON and Pump OFF buttons. Observe the relay indicator, LED, pump operation and displayed status.

Action	Relay	LED	Pump	Dashboard
Pump ON				
Pump OFF				

Timed Control

Use the timed-pump function to run the pump for a safe duration. You may also test the route through /docs.

```
POST /api/pump/timed
```

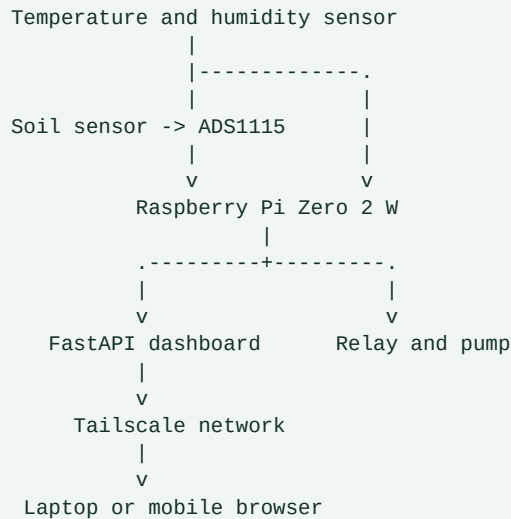
```
{
  "duration": 3
}
```

Duration	Pump started	Stopped automatically
1 second		
3 seconds		
5 seconds		

Safety

Use only pump durations approved for the training setup. If the pump does not stop, disconnect the external pump power immediately.

Complete Prototype Integration



Full-System Test

Task	Completed
I2C devices are detected.	
FastAPI starts successfully.	
Dashboard opens through Tailscale.	
Temperature and humidity are displayed.	
Soil-moisture percentage is displayed.	
Manual pump ON works.	
Manual pump OFF works.	
Timed pump control works.	
Pump stops automatically.	
Pump remains off after FastAPI stops.	
API documentation opens.	

Practice Challenges

Change the refresh rate

Open `static/app.js` and modify `setInterval(updateReadings, 3000)`. The value is in milliseconds.

Change the pump duration

Modify the default duration sent by the dashboard and test the new safe value.

Add a system message

Display `Soil is dry` when the moisture level is below the threshold and `Soil moisture is sufficient` when it is above the threshold.

Troubleshooting Guide

Git or repository problem

```
sudo apt install -y git
git --version
git status
```

Virtual environment or missing package

```
cd ~/iot-pi-training/dashboard
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

FastAPI does not start

```
pwd
ls
uvicorn main:app --host 0.0.0.0 --port 8000
```

Port 8000 is already in use

```
ss -ltnp | grep 8000
uvicorn main:app --host 0.0.0.0 --port 8001
```

Dashboard cannot be opened through Tailscale

```
tailscale status
tailscale ip -4
curl http://$(tailscale ip -4):8000/health
```

Sensor values are missing

```
i2cdetect -y 1
curl http://127.0.0.1:8000/api/readings
```

Pump does not stop

Immediate action

Disconnect the external pump power. Check that the relay uses NO rather than NC, verify the active state, and confirm the shutdown logic turns the relay off.

Module Completion Check

Task	
Git is installed and the approved repository is cloned.	
Project structure is inspected and the virtual environment is active.	
Dependencies and hardware values are configured.	
FastAPI health and sensor routes are tested.	
The dashboard opens through the authorised Tailscale connection.	
Temperature, humidity and soil moisture are displayed.	
Manual and timed pump controls are tested safely.	
The pump stops safely and remains OFF after application exit.	

Module Outcome

The group has a working browser-based IoT dashboard with live sensing, bounded pump control, GitHub-managed source code and authorised private-network access.

Final Prototype Record

Information	Your record
Group number	
Raspberry Pi hostname	
Tailscale IP address	
Dashboard address	
SHT31 address	
ADS1115 address	

Information	Your record
Relay active state	
Safe pump duration	
GitHub repository	

Training Completion Reflection

The most useful skill I learned was:

The most difficult part of the prototype was:

The problem was solved by:

One feature I would add to the system is:

A possible real-world application for this prototype is: